1

평가참여

평가 평가참여

학습단계
AI 도구 학습

학습주제
Python과 Git 심화 (Python & Git advanced

미션
친구 3~5명과 함께 프로그램 만드는 법 연습하기

탐험가명
노신용,김세윤,김준원,방승규

1. 학습단계 소개

AI 도구 학습 팀정보 노신용 김세윤 김준원 방승규

기술적 설명

운영체제, 자료구조, 웹, 데이터베이스, 클라우드까지!
소프트웨어를 이루는 핵심 기술을 직접 구현하며 익히는 단계입니다.
각 기술을 따로따로 배우는 것이 아니라, 하나의 서비스를 만들어가는 과정에서 자연스럽게 연결합니다.
이 단계를 마치면 서비스 전체 구조를 이해하고 프로그램을 개발할 수 있는 주니어 개발자로 성장합니다.

개발환경

N/A

제약조건

N/A

Test case

N/A

2. 학습주제 소개

Python과 Git 심화 (Python & Git advanced)

기술적 설명

파일 입출력, 클래스, 모듈 분리를 활용해 유지보수 가능한 콘솔 프로그램을 만듭니다.

제너레이터, 데코레이터, 타입 힌트 등 실무 수준의 코딩을 경험하고, GitHub Flow 기반의 PR·코드리뷰·충돌 해결까지 팀 협업 워크플로우를 실전으로 익힙니다.

개발환경

N/A

제약조건

N/A

Test case

N/A

3. 미션 소개

친구 3~5명과 함께 프로그램 만드는 법 연습하기

기술적 설명



1. 미션 소개

Git은 단순한 버전 관리 도구가 아니라, 팀 협업의 핵심 인프라입니다. 아무리 코딩을 잘해도 Git을 제대로 다루지 못하면 팀 프로젝트에서 병목이 됩니다.

이 미션에서는 3~5인 팀으로 실제 협업 상황을 시뮬레이션합니다. 같은 파일을 동시에 수정하여 충돌을 경험하고, PR로 코드 리뷰를 주고받으며, 브랜치 전략을 적용합니다. 단순히 명령어를 외우는 것이 아니라, "왜 이 워크플로우가 필요한지"를 팀원들과 함께 체득합니다.

이 미션의 경험은 실무에서 바로 적용됩니다. 특히 PR 기반 협업과 코드 리뷰 문화는 현업에서 가장 중요한 소프트 스킬 중 하나입니다.

2. 최종 결과물

다음 산출물을 포함한 팀 GitHub 저장소 1개를 완성한다.

- GitHub 저장소 URL
- 제출물 인덱스 문서: SUBMISSION.md (팀원별 PR 링크/문서 링크/증빙 링크를 한 곳에 모음)
- 협업 문서 3종
    - docs/CONTRIBUTING.md
    - docs/conflict-resolution.md
    - docs/troubleshooting-log.md
- Git 히스토리 증빙(택 1)
    - git log --oneline --graph --all 결과 텍스트(또는 스크린샷)
- “간단한 결과물”(아래 중 택 1)
    - 팀원별 유틸 함수 모음(언어 자유 / 예: Python)
    - 팀 소개 문서(README + team 폴더)

    - 학습 정리 노트(팀원별 마크다운)

3. 과제 목표

이 과제를 마친 후, 학습자는 아래를 스스로 설명할 수 있어야 한다.

- 브랜치가 내부적으로 어떻게 동작하는지(커밋을 가리키는 포인터), 그리고 왜 브랜치를 나눠서 작업하는지 설명할 수 있다.
- GitHub Flow가 무엇이고, 왜 팀 협업에서 이런 전략이 필요한지 설명할 수 있다.
- PR(Pull Request)의 목적과 코드 리뷰의 가치를 설명할 수 있다.
- 충돌이 왜 발생하고, 충돌 마커(`<<<<<<<` , `=======` , `>>>>>>>`)가 무엇을 의미하는지 설명할 수 있다.
- `reset` , `revert` , `stash` 의 차이와 각각 언제 사용하는지 설명할 수 있다.
- 팀 협업 중 문제가 생겼을 때(충돌/실수 커밋 등) 기록을 남기고 재현 가능한 방식으로 해결 과정을 설명할 수 있다.

4. 기능 요구 사항

다음 요구사항을 모두 만족해야 한다.

1. 팀 구성 및 저장소 준비

    - 3~5인 팀으로 진행한다.
    - 저장소는 아래 중 하나로 구성한다.

        - 옵션 A(권장): GitHub Organization 저장소 생성
        - 옵션 B(대체): 개인 저장소 생성 후 Collaborator 초대

    - 기본 폴더/파일을 구성한다.

        - `README.md`
        - `docs/` (문서)
        - `src/` (코드 또는 예시 파일)
        - `team/` (팀원별 소개 문서 선택 시 사용)

    - Branch Protection Rule을 main 브랜치에 설정한다.

        - main 직접 push 금지
        - PR을 통한 병합만 허용
        - 최소 1명 승인(approve) 필요

2. 브랜치 전략(GitHub Flow) 적용

- GitHub Flow를 적용한다.
  - main : 항상 배포 가능한 상태(팀 기준에서 “깨지지 않는 상태”)
  - feature/* : 작업 단위 브랜치
- 브랜치 네이밍 규칙을 정하고 docs/CONTRIBUTING.md 에 문서화한다.
- GitHub Flow를 선택한 이유를 3줄 이내로 README.md 또는 docs/CONTRIBUTING.md 에 기록한다.

3. 이슈 기반 작업 및 PR 연동(필수)

- 각 작업은 Issue로 생성하고, PR과 연동한다.
- PR 본문에 Closes #이슈번호 (또는 Fixes #)를 포함하여 이슈가 추적 가능해야 한다.
- SUBMISSION.md 에서 팀원별로 “내가 만든 Issue/PR”을 빠르게 확인할 수 있어야 한다.

4. 커밋 메시지 컨벤션

- 팀 커밋 메시지 규칙을 정하고 docs/CONTRIBUTING.md 에 문서화한다.
- 권장 형식(예시):
  - feat: subject , fix: subject , docs: subject , refactor: subject 등
- 의미없는 커밋 메시지 금지(판정 기준) 아래 중 하나라도 해당하면 “의미없는 메시지”로 간주한다.
  - 변경 대상을 유추할 수 없는 단어만 있는 경우: update , fix , temp , wip , final 등
  - 무엇을/왜 바꿨는지 드러나지 않는 경우: bug fix , edit file 등(구체 대상/효과 없음)

5. PR 기반 협업(팀원별 기여도 기준 포함)

- 모든 feature 브랜치는 PR로 main에 병합한다.
- 팀원별 최소 기준(각 팀원 기준, 전원 충족):
  - PR 생성 및 병합: 최소 2개
  - 코드 리뷰 작성: 최소 2개(본인 PR 제외)
  - 리뷰 반영 경험: 본인 PR에서 최소 1회 이상 리뷰 코멘트를 반영(커밋/수정/답글로 증빙)

- PR 본문에는 최소한 다음이 포함되어야 한다(형식은 자유).
    - 변경 사항(What)
    - 변경 이유(Why)
    - 테스트/검증 방법(How)

6. 코드 리뷰 최소 품질 기준

- 각 PR에는 “LGTM/좋아요”만이 아닌 실질 코멘트 1개 이상이 포함되어야 한다.
    - 예: 특정 라인/파일을 근거로 질문, 대안 제안, 리스크 지적, 개선 제안
- 리뷰어와 작성자 간 최소 1회 이상 상호작용(답글/수정 반영)이 기록으로 남아야 한다.

7. 충돌 해결 실습(비자명 충돌 포함)

- 의도적으로 충돌 상황을 만들고 해결한다.
- 팀 전체 기준: 최소 2회 이상 충돌 해결 기록이 남아야 한다(커밋/PR/문서).
- 최소 1회는 “비자명 충돌”이어야 한다(둘 중 하나 충족).
    - 같은 파일의 같은 hunk(인접 라인) 를 서로 다르게 수정하여 충돌
    - 한쪽은 파일 이동/이름 변경(또는 삭제), 다른 한쪽은 내용 수정(충돌 또는 리베이스/머지 시 이슈 유발)
- 충돌 해결 과정은 `docs/conflict-resolution.md` 에 기록한다(템플릿 준수 권장).

8. Git 트러블슈팅 실습(팀 전체 4종 + 팀원 참여)

- 아래 4가지 시나리오는 팀 전체가 모두 수행하고 문서로 남긴다.
    a. `git commit --amend` (최근 커밋 메시지 수정)
    b. `git reset --soft HEAD~1` (로컬 커밋 취소 + 변경 유지)
    c. `git revert` (원격에 push된 커밋 취소)
    d. `git stash` / `git stash pop` (작업 보관 후 전환)
- 팀원별 최소 기준: 각 팀원은 최소 1개 시나리오의 해결 기록 작성에 참여해야 한다(이름/역할이 문서에 드러나야 함).
- 모든 기록은 `docs/troubleshooting-log.md` 에 남긴다.

9. 협업 가이드 문서 작성

- docs/CONTRIBUTING.md 에 팀의 협업 규칙을 정리한다(팀원이 분담하여 작성).
- 포함 내용(최소):
  - 브랜치 네이밍 규칙
  - 커밋 메시지 컨벤션
  - PR 작성 규칙(본문에 포함할 항목)
  - 코드 리뷰 규칙(리뷰 최소 품질 기준 포함)
  - 충돌 발생 시 기본 대응 흐름(누가/어떻게/어디에 기록)

10. 간단한 결과물(택 1) + 최소 완성 기준

- 아래 중 하나를 선택한다(복잡한 기능 구현이 목적이 아님).
  - (A) 유틸 함수 모음: 팀원별 함수 1개 이상 + 간단 사용 예시(README 또는 docstring)
  - (B) 팀 소개: team/ 에 팀원별 소개 파일 + README에서 링크/목록 제공
  - (C) 학습 노트: 팀원별 노트 1개 이상 + README에서 목차 제공
- 팀원별 최소 기준: 선택한 결과물에 대해 각 팀원은 최소 1건의 기여 커밋이 있어야 한다.

5. 보너스 과제 (선택)

1. 히스토리 정리
   - git rebase -i 로 개인 feature 브랜치의 커밋을 squash/reword하여 정리한다.
   - 정리 전/후 히스토리를 비교해 간단히 문서화한다.
2. CODEOWNERS/리뷰어 자동화
   - .github/CODEOWNERS 를 추가해 파일/폴더별 책임 리뷰어를 지정한다.

개발환경

6. 개발 환경

- Python 3.10 이상

- Git , GitHub 사용

제약조건

7. 제약 사항

- 팀 구성
  - 3~5인 팀으로 진행
  - 모든 팀원이 Git 작업에 참여해야 함 (팀원별 최소 기준은 기능 요구 사항 참조)
- 핵심 목표
  - 복잡한 기능 구현이 아닌, Git 협업 워크플로우 학습이 목적
  - 결과물 퀄리티보다 협업 과정(브랜치, PR, 리뷰, 이슈, 충돌 해결, 기록) 이 더 중요
- 플랫폼
  - GitHub 사용
- 운영 안정성(중요)
  - 공유 브랜치(main 포함)에서 무리한 히스토리 재작성 금지
  - 팀 합의 없이 강제 푸시/리베이스 금지
  - 충돌/트러블슈팅은 “재현 가능한 기록”이 남아야 함 (문서에 상황/절차/결과/주의점 포함)

Test case

8. 결과 예시

아래는 정답이 아니라 참고 예시다. 실제 문구/구조/디자인은 달라도 된다.

- 브랜치 히스토리 예시

```
* (main) Merge pull request #8 from feature/lee-string-utils
|\\
```

```
| * feat: add string utils (apply review feedback)
|/
* Merge pull request #7 from feature/kim-math-utils
|\\
| * feat: add math utils
|/
* Merge pull request #6 from feature/park-contributing
|\\
| * docs: add contributing guide
|/
* Initial commit
```

- PR 본문 템플릿 예시(형식 자유)

```
제목: feat: <작업 요약>

## 연결 이슈
- Closes #<issue_number>

## 변경 사항(What)
- ...

## 변경 이유(Why)
- ...

## 테스트/검증(How)
- [ ] 로컬 실행/간단 테스트
- [ ] 충돌 가능성 체크(필요 시)
```

- docs/conflict-resolution.md 템플릿 예시

```
# Conflict Resolution Log

## 충돌 기록 #1
### 참여자
- 작성자: <name>
- 상대: <name>
```

````
### 상황(What happened)
- 어떤 브랜치/파일에서 충돌이 났는지

### 충돌 내용(Conflict markers)
```txt
<<<<<<< HEAD
...
=======
...
>>>>>>> feature/...
```

### 해결 과정(How)
- 선택한 해결 전략(keep both / choose one / refactor)과 이유
- 실제로 수행한 명령 또는 절차

### 결과(Outcome)
- 최종 병합 결과 요약, 관련 PR/커밋 링크

### 배운 점(Learnings)
- 다음에 예방하려면 무엇을 할지
````

- docs/troubleshooting-log.md 템플릿 예시

```css
# Troubleshooting Log

## 시나리오: <amend/reset/revert/stash>
### 참여자
- <name>, <name>

### 상황
- 문제가 무엇이었는지(재현 가능한 설명)

### 시도한 명령/절차
- (예) git ...

### 결과
- 무엇이 어떻게 해결됐는지
```

- 주의할 점(특히 원격 히스토리/협업 영향)

왜 이 방법을 선택했는가(Why)
- reset 대신 revert를 선택한 이유 등

- docs/CONTRIBUTING.md 템플릿 예시

```CSS
# Contributing Guide

## 브랜치 전략(GitHub Flow)
- main / feature/* 규칙
- (3줄) 우리 팀이 GitHub Flow를 선택한 이유: ...

## 브랜치 네이밍 규칙
- 예: feature/<name>-<topic>

## 커밋 메시지 컨벤션
- 예: feat: ..., fix: ...
- 금지 예: update, fix(대상 불명), wip 등

## PR 규칙
- PR 본문 필수 항목(What/Why/How + Closes #issue)
- 병합 조건(리뷰 1명 승인 등)

## 코드 리뷰 규칙
- “LGTM만 금지”, 라인 근거 코멘트 예시

## 충돌 대응 흐름
- 충돌 발생 → 공유 → 해결 → 기록(conflict-resolution.md)
```

- SUBMISSION.md 예시 (제출물 인덱스)

```CSS
# Submission Index

## Team
- 팀명: ...
- 저장소: <repo_url>
```

```
## Member PRs
- <name1>
  - PR: <url>, <url>
- <name2>
  - PR: <url>, <url>
- ...

## Key Docs
- Contributing: docs/CONTRIBUTING.md
- Conflict log: docs/conflict-resolution.md
- Troubleshooting: docs/troubleshooting-log.md

## Evidence
- git log screenshot or text: <url_or_paste_location>
```

4. 평가자료

프로젝트 URL

https://github.com/Beda28/B2-2.Team

Branch명

main

5. 평가문항

항목 1

- (옵션 A/B) 저장소가 팀 단위로 1개이며, 팀원이 협업 가능한 권한으로 참여했는가?
- main 브랜치에 Branch Protection이 설정되어 있고, 직접 push 없이 PR로만 병합되었는가?
- 각 PR이 이슈와 연동되어 있으며(예: Closes #n), 추적 가능한가?
- 팀원별로 PR 2개 이상이 병합되어 있고, 팀원별 PR 링크를 SUBMISSION.md 에서 확인할 수 있는가?

PASS

- 팀원별로 리뷰 2개 이상이 존재하고, 각 팀원이 최소 1회는 본인 PR에서 리뷰를 반영했는가?
- 충돌 해결 기록이 최소 2회 존재하며, 그 중 1회는 비자명 충돌 기준을 충족하는가?
- 트러블슈팅 4종(amend/reset/revert/stash)이 모두 수행되었고, 팀원별로 최소 1개 시나리오 기록에 참여했는가?
- docs/CONTRIBUTING.md , docs/conflict-resolution.md , docs/troubleshooting-log.md 가 존재하고 템플릿 수준의 구조를 갖추었는가?

FAIL

항목 2

- 브랜치 전략을 “작업 단위”로 나누는 기준을 어떻게 정했는지 설명할 수 있는가?
- PR 본문에 What/Why/How와 이슈 연동 정보를 일관되게 남기기 위해 어떤 규칙/도구(템플릿 등)를 썼는지 설명할 수 있는가?
- 리뷰 코멘트가 “LGTM”에 그치지 않도록 팀에서 적용한 최소 품질 기준과 운영 방법을 설명할 수 있는가?
- 충돌 발생 시, 팀이 어떤 흐름(공유→해결→기록)으로 대응했는지 설명할 수 있는가?
- 트러블슈팅 로그를 “재현 가능”하게 만들기 위해 어떤 항목(상황/절차/결과/주의점)을 고정했는지 설명할 수 있는가?

PASS

FAIL

항목 3

- GitHub Flow에서 main을 “항상 배포 가능 상태”로 유지해야 하는 이유를 설명할 수 있는가?
- main에 직접 push하지 않고 PR + 승인으로 병합하는 이유(품질/책임/추적성)를 설명할 수 있는가?
- 이슈-PR 연동을 하는 이유(추적성/커뮤니케이션/자동화)를 설명할 수 있는가?
- 원격에 push된 커밋을 되돌릴 때 reset 대신 revert를 선택해야 하는 이유를 설명할 수 있는가?
- 충돌 마커(<<<<<<< , ======= , >>>>>>>)의 의미와, 비자명 충돌을 해결할 때 고려한 의사결정 기준을 설명할 수 있는가?

PASS

FAIL

항목 4

- 만약 main에 긴급 핫픽스가 필요하다면, GitHub Flow에서 어떤 순서로 처리할지(브랜치/PR/리뷰/배포) 설명할 수 있는가?
- 만약 팀원이 실수로 “의미없는 커밋 메시지”를 여러 개 push했다면, 협업 중단 없이 개선하는 방법을 설명할 수 있는가?
- 만약 충돌이 반복적으로 발생한다면(같은 파일/같은 영역), 원인 분석과 예방 방안을 제시할 수 있는가?
- (보너스 수행 시) 왜 리베이스가 히스토리를 깨끗하게 만들지만 협업에서는 위험할 수 있는지, 안전 수칙을 포함해 설명할 수 있는가?

○ PASS
○ FAIL

항목 5

보너스 문제 해결에 따른 크레딧 부여 ○ 부여 ○ 미부여

6. 평가결과

평가 시작일시: 2026-10-11 13:55

평가 결과: FAIL

FAIL

경험정도

○ 재구현 불가

평가 피드백

ⓘ 평가 피드백 등록을 완료하여야 평가 확정이 가능합니다.

내용

최소 100자 이상 입력해주세요 (현재 0자)

0 / 2000

평가 피드백 우수가이드 보기

목록 저장